PCCST403 OPERATING SYSTEMS

MODULE 2

Concurrency and Synchronization - Basic principles, Mechanisms - Locks: The Basic Idea, Building Spin Locks with Test-And-Set, Compare and Swap, Using Queues: Sleeping Instead of Spinning, Semaphores - Definition, Binary Semaphores, The Producer/Consumer (Bounded Buffer) Problem and its solution using semaphores, Reader-Writer Locks.

Concurrency: Deadlock and Starvation - Deadlock Characterization, Deadlock Prevention and Avoidance, Deadlock Detection and recovery, Dining Philosophers Problem and its solution.

****

PROCESS SYNCHRONIZATION

On the basis of synchronization, processes are categorized as one of the following two types:

- Independent Process
- Cooperative Process

Process synchronization problem arises in the case of Cooperative process because resources are shared there.

Process synchronization is the task or phenomenon of coordinating the execution of processes in such a way that no two processes can have access to the same shared data and resources.

- It is a procedure that is involved in order to preserve the appropriate order of execution of cooperative processes. In order to synchronize the processes, there are various synchronization mechanisms.
- Process Synchronization is mainly needed in a multi-process system when multiple processes are running together, and more than one processes try to gain access to the same shared resource or any data at the same time.

RACE CONDITION

- A race condition is a problem that occurs in an operating system where two or more processes or threads are executing concurrently. The outcome of their execution depends on the order in which they are executed. In a race condition, the exact timing of events is unpredictable, and the outcome of the execution may vary based on the timing. This can result in unexpected or incorrect behaviour of the system.

For example,

- If two threads are simultaneously accessing and changing the same shared resource, such as a variable or a file, the final state of that resource depends on the order in which the threads execute. If the threads are not correctly synchronized, they can overwrite each other's changes, causing incorrect results or even system crashes.

Effects of Race Condition in OS:

In an operating system, a race condition can have several effects, which are:

1. Deadlocks: A race condition can cause deadlocks, where two or more processes are awaiting the completion of a job by one another but are unable to move forward, can be brought on by a race condition. This can happen if two processes try to access the same resource simultaneously, and the operating system doesn't have a mechanism to ensure that only one can access the resource at a time.

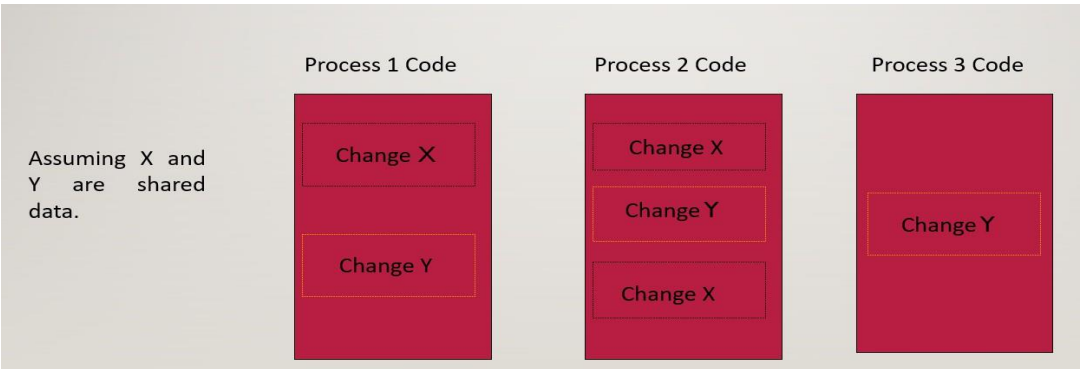
2. **Data corruption:** A race condition can cause data corruption, where two or more processes simultaneously try to write/update the exact memory location. This can cause the data to be

overwritten or mixed up, resulting in incorrect results or program crashes.

3. **Security vulnerabilities**: A race condition can also create security vulnerabilities in an operating system. For example, an attacker may be able to exploit a race condition to gain unauthorized access to a system or to escalate their privileges.
4. **Performance degradation:** In some cases, a race condition can also cause performance degradation in an operating system. This can happen if multiple processes are competing for the same resources, such as CPU time or memory, and the operating system is unable to allocate these resources efficiently.

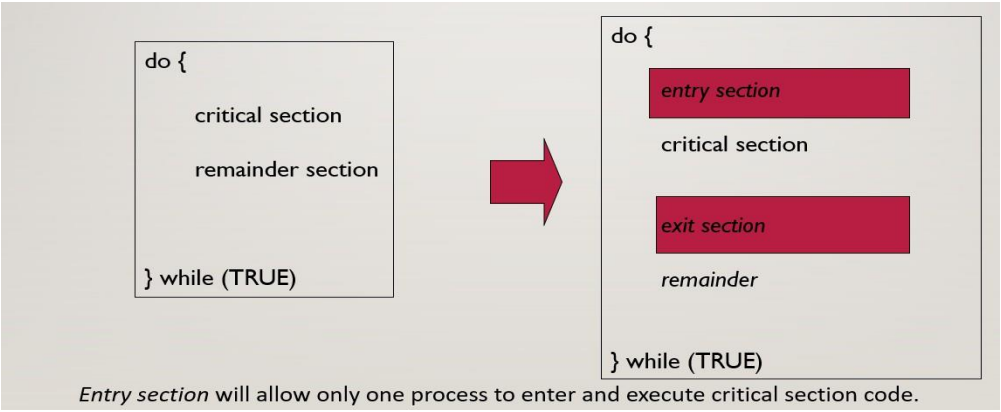
PROGRAMS AND CRITICAL SECTIONS

The part of the program (process) that is accessing and changing shared data is called its critical section.



Critical Section

- **The general way to do that is:**



- Consider system of *n* processes $\{p_0, p_1, \ldots p_{n-1}\}$
- Each process has critical section segment of code
  - Process may be changing common variables, updating table, writing file, etc
  - When one process in critical section, no other may be in its critical section
- *Critical section problem* is to design protocol to solve this.
- Each process must ask permission to enter critical section in entry section, may follow critical section with exit section, then remainder section

- ENTRY SECTION:- The process will request to enter the critical section then a part of code decide wheather process can enter in the **Critical Section** on not.
- CRITICAL SECTION:- A code segment that accesses shared resources and that has to be executed as an atomic action.
- EXIT SECTION:- Locking section can be undone which is done in **Critical Section**.
- REMAINDER SECTION:- Remaining part of the program .

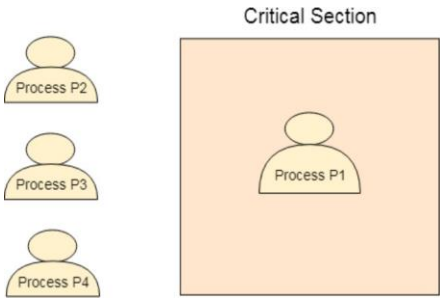
Critical Section is the part of a program which tries to access shared resources. That resource may be any resource in a computer like a memory location, data structure, CPU or any IO device. The critical section cannot be executed by more than one process at the same time; operating system faces the difficulties in allowing and disallowing the processes from entering the critical section.

The critical section problem is used to design a set of protocols which can ensure that the race condition among the processes will never arise.

Solution to Critical Section Problem

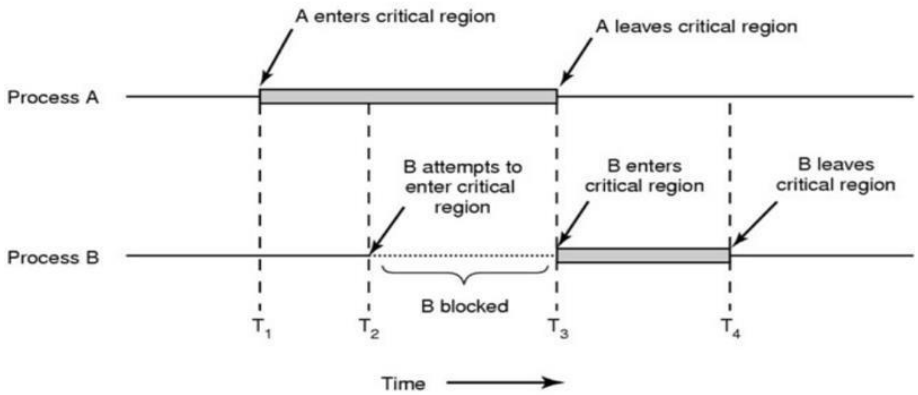
There are 3 requirements that stand for a correct solution:

1. **Mutual Exclusion:** If one process is executing inside critical section then the other process must not enter in the critical section



Only one process can be executed at a time. That means access to the critical section must be mutually exclusive.

Region of code that two or more related processes *must not execute at the same time* in order to avoid race-conditions.



2. **Progress :** If no process is executing in its critical section and there exist some processes that wish to enter their critical section , then the selection of the process that will enter the critical section next cannot be postponed indefinitely.

   That is, if one process doesn't need to execute into critical section then it should not stop other processes to get into the critical section. If only one process wants to enter, it should be able to enter .If two or more want to enter, one of them should succeed.

3. **Bounded Waiting :** A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

   That is, we should be able to predict the waiting time for every process to get into the critical section. The process must not be endlessly waiting for getting into the critical section.

Critical-Section Handling in OS

Two approaches depending on if kernel is preemptive or non- preemptive

a. **Preemptive** – allows preemption of process when running in kernel mode.
b. **Non-preemptive** – runs until exits kernel mode, blocks, or voluntarily yields CPU
   i. Essentially free of race conditions in non-preemptive kernel mode

MECHANISMS

LOCKS

A **lock** is a synchronization mechanism used in concurrent programming to ensure **mutual exclusion** — meaning only one thread or process can access a shared resource (critical section) at a time.
When multiple threads access shared data simultaneously, we can get **race conditions**, leading to incorrect results.
If two threads do this at the same time, updates can be lost.

Locks prevent this by enforcing:

- Only one thread enters the critical section
- Others must wait

A **spin lock** is a lock where threads wait in a loop ("spin") until the lock becomes available.

Properties of Spin Locks

✅ Advantages

- Simple
- Fast when critical section is short
- No context switch overhead

❌ Disadvantages

- Wastes CPU while spinning
- Poor performance under contention
- Not fair (no guarantee of order)

Hardware Based Solutions

Two algorithms in the hardware approach of solving Process Synchronization problem are:

1. Test and Set
2. Compare and Swap

1. Test and Set:

Here, the shared variable is lock which is initialized to false. TestAndSet(lock) algorithm works in this way :

It always returns whatever value is sent to it and sets lock to true. The first process will enter the critical section at once as TestAndSet(lock) will return false and it'll break out of the while loop. The other processes cannot enter now as lock is set to true and so the while loop continues to be true. Mutual exclusion is ensured. Once the first process gets out of the critical section, lock is changed to false. So, now the other processes can enter one by one. Progress is also ensured. However, after the first process, any process can go in. There is no queue maintained, so any new process that finds the lock to be false again can enter. So bounded waiting is not ensured.

```
do {
 while (TestAndSet (&lock) ) ;
// do nothing
// critical section
lock = FALSE;
// remainder section
} while (TRUE);
```

Test and Set Pseudocode –

```
do {
        acquire lock
        critical section
        release lock

remainder section

} while (TRUE);
```

TestAndSet()

Definition:

```
boolean TestAndSet (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv:
}
```

1. Executed atomically
2. Returns the original value of passed parameter
3. Set the new value of passed parameter to "TRUE".

Modern machines provide **special atomic hardware instructions**

Atomic = non-interruptible

When first thread enters critical section, other threads keep spinning in loop.
The three axis of evaluating spin locks is correctness, fairness and performance.

2. Compare and swap()

The compare and swap() instruction, in contrast to the test and set() instruction, operates on three operands; The operand value is set to new value only if the expression (*value == expected) is true. Regardless, compare and swap() always returns the original value of the variable value. Like the test and set() instruction, compare and swap() is executed atomically.

```
int compare_and_swap(int *value, int expected, int new_value) {
   int temp = *value;

   if (*value == expected)
      *value = new_value;

   return temp;
}
```

```
do {
   while (compare_and_swap(&lock, 0, 1) != 0)
      ; /* do nothing */

      /* critical section */

   lock = 0;

      /* remainder section */
} while (true);
```

Mutual exclusion can be provided as follows: a global variable (lock) is declared and is initialized to 0. The first process that invokes compare and swap() will set lock to 1. It will then enter its critical section, because the original value of lock was equal to the expected value of 0. Subsequent calls to compare and swap() will not succeed, because lock now is not equal to the expected value of 0. When a process exits its critical section, it sets lock back to 0, which allows another process to enter its critical section. Although these algorithms satisfy the mutual-exclusion requirement, they do not satisfy the bounded-waiting requirement.

Using Queues: Sleeping Instead Of Spinning

The real problem with spinning approach is that they leave too much to chance. The scheduler determines which thread runs next; if the scheduler makes a bad choice, a thread runs that must either spin waiting for the lock, or yield the CPU immediately. Either way, there is potential for waste and no prevention of starvation. Thus, we must explicitly exert some control over who gets to acquire the lock next after the current holder releases it. To do this, we will need a little more OS support, as well as a queue to keep track of which threads are waiting to enter the lock.

Example: In Solaris, there are three calls:

- park() to put a calling thread to sleep
- unpark(threadID) to wake a particular thread as designated by threadID.

These two routines can be used in tandem to build a lock that puts a caller to sleep if it tries to acquire a held lock and wakes it when the lock is free.

- setpark():- By calling this routine, a thread can indicate it is about to park. If it then happens to be interrupted and another thread calls unpark before park is actually called, the subsequent park returns immediately instead of sleeping.

SEMAPHORE

- Synchronization tool to solve critical section problem. Proposed by Edsger Dijkstra.It manages concurrent processes by using a simple integer value which is known as a semaphore.
- It provides more sophisticated ways (than Mutex locks) for process to synchronize their activities.
- Semaphore ***S*** – integer variable(shared variable)
- Can only be accessed via two indivisible (atomic) operations
  - **wait()** and **signal()**
    - Originally called **P()** and **V() which means 'to test' and 'to increment' respectively.**
- Definition of the **wait() operation**

```
wait(S) {
   while (S <= 0); // do no operation
```

Definition of the **signal() operation**

```
signal(S) {
   S++;

}
```

- **Counting semaphore** – integer value can range over an unrestricted domain. It is used to control access to a resource that has multiple instances.
- **Binary semaphore** – integer value can range only between 0 and 1
  - Same as a **mutex lock**
- Can solve various synchronization problems.

```
do
{
    Wait(S);
    //Critical Section

    Signal(S)

    //remainder section
}while(true)
```

S=1, initially

```
wait(S) {
   while (S <= 0); // do no operation
   S--;
}
```

S=0

```
signal(S) {
   S++;
}
```

S=1

Semaphore Implementation

- Must guarantee that no two processes can execute the **wait()** and **signal()** on the same semaphore at the same time. Thus, the implementation becomes the critical section problem where the **wait** and **signal** code are placed in the critical section
  - Could now have **busy waiting** in critical section implementation
    - But implementation code is short
    - Little busy waiting if critical section rarely occupied
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution

Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
  - value (of type integer)
  - pointer to next record in the list
- Two operations:
  - **block** – place the process invoking the operation on the appropriate waiting queue
  - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue

Synchronization Problems

1. PRODUCER-CONSUMER PROBLEM

The Producer-Consumer problem is a classical multi-process synchronization problem, that is we are trying to achieve synchronization between more than one process.

There is one Producer in the producer-consumer problem, Producer is producing some items, whereas there is one Consumer that is consuming the items produced by the Producer. The same memory buffer is shared by both producers and consumers which is of fixed-size.

The task of the Producer is to produce the item, put it into the memory buffer, and again start producing items. Whereas the task of the Consumer is to consume the item from the memory buffer.

- The producer should produce data only when the buffer is not full. In case it is found that the buffer is full, the producer is not allowed to store any data into the memory buffer.
- Data can only be consumed by the consumer if and only if the memory buffer is not empty. In case it is found that the buffer is empty, the consumer is not allowed to use any data from the memory buffer.
- Accessing memory buffer should not be allowed to producer and consumer at the same time.

In the code, mutex, empty and full are semaphores. Here mutex is initialized to 1, empty is initialized to n (maximum size of the buffer) and full is initialized to 0.
The mutex semaphore ensures mutual exclusion. The empty and full semaphores count the number of empty and full spaces in the buffer.
After the item is produced, wait operation is carried out on empty. This indicates that the empty space in the buffer has decreased by 1. Then wait operation is carried out on mutex so that consumer process cannot interfere. After the item is put in the buffer, signal operation is carried out on mutex and full. The former indicates that consumer process can now act and the latter shows that the buffer is full by 1.

Producer Process

The code that defines the producer process is given below –

```
do {
  .
  . PRODUCE ITEM
  .
  wait(empty);
  wait(mutex);
  .
  . PUT ITEM IN BUFFER
  .
  signal(mutex);
  signal(full);

} while(1);
```

The wait operation is carried out on full. This indicates that items in the buffer have decreased by 1. Then wait operation is carried out on mutex so that producer process cannot interfere.

Then the item is removed from buffer. After that, signal operation is carried out on mutex and empty. The former indicates that consumer process can now act and the latter shows that the empty space in the buffer has increased by 1.

Consumer Process

The code that defines the consumer process is given below:

```
do {

  wait(full);
  wait(mutex);
  . .
  . REMOVE ITEM FROM BUFFER
  .
  signal(mutex);
  signal(empty);
  .
  . CONSUME ITEM
  .
} while(1);
```

2. THE READERS–WRITERS PROBLEM

- Suppose that a database is to be shared among several concurrent processes.
- Some of these processes may want only to read the database(reader), whereas others may want to update the database(writer).
- If two readers access the shared data simultaneously, no adverse effects will result.
- If a writer and some other process (either a reader or a writer) access the database simultaneously, chaos may arise. We require that the writers have exclusive access to the shared database while writing to the database. This synchronization problem is referred to as the readers–writers problem.

Solution to The Readers–Writers Problem using Semaphores

- Make use of two semaphores and a shared integer variable. semaphore rw_mutex = 1; semaphore mutex = 1; int read_count = 0;
- The semaphores mutex and rw_mutex are initialized to 1; read_count is initialized to 0. The semaphore rw_mutex is common to both reader and writer processes.
- The mutex semaphore is used to ensure mutual exclusion when the variable read_count is updated.
- The read_count variable keeps track of how many processes are currently reading the object.
- The semaphore rw_mutex functions as a mutual exclusion semaphore for the writers.
- It is also used by the first or last reader that enters or exits the critical section.

- The code for a writer process is

```
do {
   wait(rw_mutex);
      . . .
   /* writing is performed */
      . . .
   signal(rw_mutex);
} while (true);
```

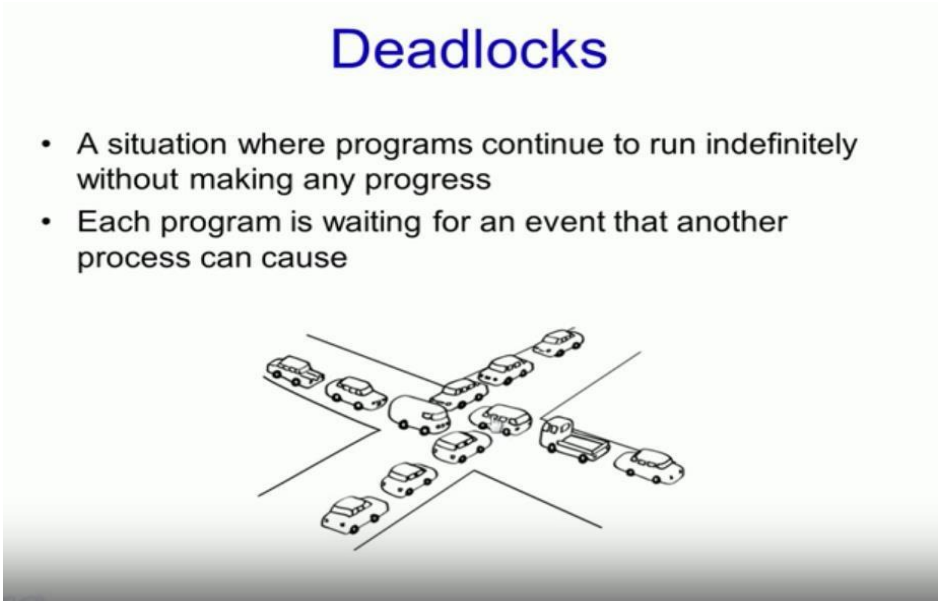
- The code for a reader process is

```
do {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed */
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
} while (true);
```

Starvation

Starvation (or indefinite blocking) occurs in **priority scheduling** when a low-priority process keeps waiting indefinitely because higher-priority processes continuously get the CPU.

- This problem arises in heavily loaded systems where resources are always occupied by higher-priority tasks, leaving some processes starved.
- To prevent this, operating systems use **aging**, a technique that gradually increases the priority of waiting processes, ensuring fair execution.



A process in operating system uses resources in the following way.

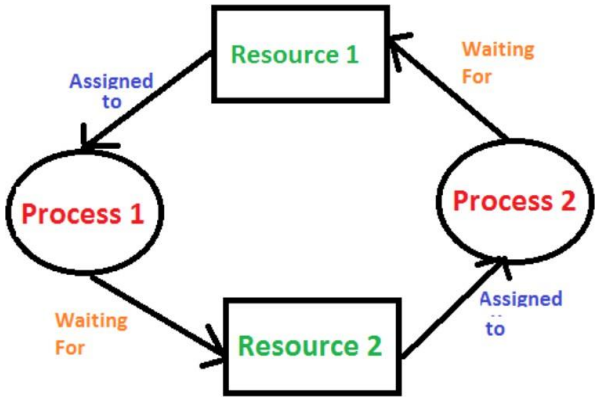
1. Requests a resource
2. Use the resource
3. Releases the resource

***A deadlock* is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.**

Consider an example when two trains are coming toward each other on the same track and there is only one track, none of the trains can move once they are in front of each other. A similar situation

occurs in operating systems when there are two or more processes that hold some resources and wait for resources held by other(s). For example, in the below diagram, Process 1 is holding Resource 1 and waiting for resource 2 which is acquired by process 2, and process 2 is waiting for resource 1.

NECESSARY CONDITIONS FOR THE OCCURRENCE OF A DEADLOCK



Following are the four conditions related to deadlock in the context of the scenario with two processes and two resources:

1. **Mutual Exclusion**

Only one process at a time can use a resource. This condition requires that at least one resource be held in a non-shareable mode, which means that only one process can use the resource at any given time. Here in this scenario, both Resource 1 and Resource 2 are non-shareable, and only one process can have exclusive access to each resource at any given time.

2. **Hold and Wait**

A process holding at least one resource is waiting to acquire additional resources held by other processes. The hold and wait condition specifies that a process must be holding at least one resource while waiting for other processes to release resources that are currently held by other processes. In the example,

- Process A has Resource 1 and is awaiting Resource 2.
- Process B currently has Resource 2 and is awaiting Resource 1.
- Both processes hold one resource while waiting for the other, satisfying the hold and wait condition.

3. **No Pre-emption**

A resource can be released only voluntarily by the process holding it, after that process has completed its task. Pre-emption is the act of taking a resource from a process before it has finished its task. According to the no pre-emption condition, resources cannot be taken forcibly from a process ; a process can only release resources voluntarily after completing its task. In this scenario, neither Process A nor Process B can be forced to release the resources in their possession. The processes can only release resources voluntarily.

4. **Circular Wait**

There exists a set {P_0, P_1, …, P_n} of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0. This condition implies that circular processes must exist, with each process waiting for a resource held by the next process in the chain. In this scenario, Process A is waiting for Resource 2, which is being held by Process B. Process B is awaiting Resource 1 from Process A. This circular chain of dependencies causes a deadlock because neither process can proceed, resulting in a system shutdown.

Deadlocks are a major concern in operating systems, and various techniques are used to avoid them to prevent, detect, and recover from them.

RESOURCE-ALLOCATION GRAPH

Deadlocks can be described more precisely in terms of a directed graph called a system resource-allocation graph. This graph consists of a set of vertices V and a set of edges E. The set of vertices V is partitioned into two different types of nodes: P = {P1, P2, ..., Pn}, the set consisting of all the active **processes** in the system, and R = {R1, R2, ..., Rm}, the set consisting of all **resource types** in the system.

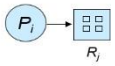
A directed edge from process Pi to resource type Rj is denoted by Pi → Rj ; it signifies that process Pi has requested an instance of resource type Rj and is currently waiting for that resource. A directed edge from resource type Rj to process Pi is denoted by Rj → Pi ; it signifies that an instance of resource type Rj has been allocated to process Pi . A directed edge Pi → Rj is called a **request edge**; a directed edge Rj → Pi is called an **assignment edge.**

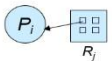
Pictorially, we represent each process Pi as a circle and each resource type Rj as a rectangle.

- Since resource type Rj may have more than one instance, we represent each such instance as a dot within the rectangle.
- Note that a request edge points to only the rectangle Rj , whereas an assignment edge must also designate one of the dots in the rectangle.
- When process Pi requests an instance of resource type Rj , a request edge is inserted in the resource-allocation graph. When this request can be fulfilled, the request edge is instantaneously transformed to an assignment edge.
- When the process no longer needs access to the resource, it releases the resource. As a result, the assignment edge is deleted.

A set of vertices *V* and a set of edges *E*. In short:

- **V is partitioned into two types:**
  - ***P* = {P_1, P_2, …, P_n}, the set consisting of all the processes in the system**
  - ***R* = {R_1, R_2, …, R_m}, the set consisting of all resource types in the system**

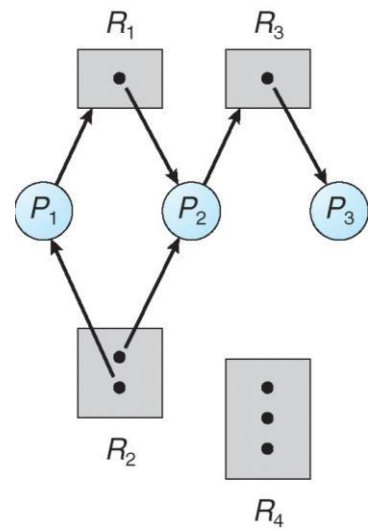
- Process
- Resource Type with 4 instances
- P_i requests instance of R_j 

- P_i is holding an instance of R_j 

- **request edge – directed edge $P_i \rightarrow R_j$**

assignment edge – directed edge $R_j \rightarrow P$

Example of a Resource Allocation Graph



Consider the graph shown here :
It represents • The sets P, R, and E:

- P = {P1, P2, P3}
- R = {R1, R2, R3, R4}
- E = {P1 → R1, P2 → R3, R1 → P2, R2 → P2, R2 → P1, R3 → P3}

Resource instances:

• One instance of resource type R1 • Two instances of resource type R2
• One instance of resource type R3 • Three instances of resource type R4 Process states:

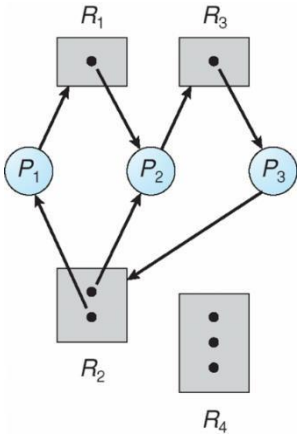
- Process P1 is holding an instance of resource type R2 and is waiting for an instance of resource type R1.
- Process P2 is holding an instance of R1 and an instance of R2 and is waiting for an instance of R3.
- Process P3 is holding an instance of R3.

Suppose that process P3 requests an instance of resource type R2.
Since no resource instance is currently available, we add a request edge P3 → R2 to the graph.

- At this point, two minimal cycles exist in the system:
- P1 → R1 → P2 → R3 → P3 → R2 → P1
- P2 → R3 → P3 → R2 → P2
- **If the graph contains no cycles, then no process in the system is deadlocked. If the graph does contain a cycle, then a deadlock may exist.**
- **If each resource type has exactly one instance, then a cycle implies that a deadlock has occurred. In this case a cycle in the graph is both a necessary and a sufficient condition for deadlock.**

If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred. In this case, a cycle in the graph is a necessary but not a sufficient condition for the existence of deadlock.

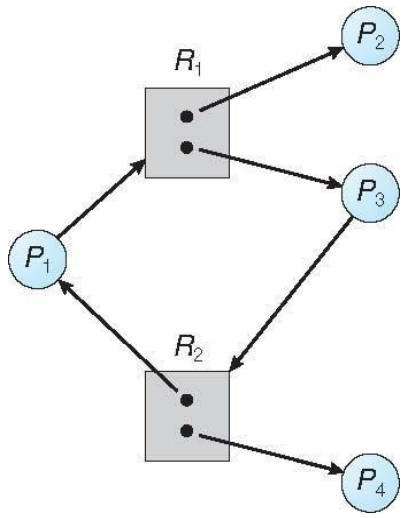
Resource Allocation Graph With A Deadlock



Here, in this graph , processes P1, P2, and P3 are deadlocked.

- Process P2 is waiting for the resource R3, which is held by process P3.
- Process P3 is waiting for either process P1 or process P2 to release resource R2.
- Process P1 is waiting for process P2 to release resource R1.

Graph With A Cycle But No Deadlock



Consider the resource-allocation graph shown here: We have a cycle: P1 → R1 → P3 → R2 → P1

- There is no deadlock.
- Observe that process P4 may release its instance of resource type R2.
- That resource can then be allocated to P3, breaking the cycle.

In summary, if a resource-allocation graph does not have a cycle, then the system is not in a deadlocked state. If there is a cycle, then the system may or may not be in a deadlocked state

Minimum Resources Required

Suppose there are three processes P1, P2 and P3. P1 requires 2 resources, P2 requires 3 resources , P3 requires 4 resources. What are the minimum no. of resources required so that system will never enter in deadlock?

Assign one less no. of resources than required to each process.

i.e. firstly assign 1->P1, 2->P2, 3->P3.

Then, to complete any of given process we simply need 1 more resource.

MINIMUM RESOURCES REQUIRED

- In general, minimum no. of resources required so that system must be in safe state:

  $\{(R_1-1)+(R_2-1)+(R_3-1)+----------+(R_n-1)\}+1$

  $(\sum R_i-n)+1$ where i is from 1 to n

$$\boxed{(\sum R_i-n)+1}$$

- Suppose there are five processes P1, P2, P3, P4 and P5. P1 requires 4 resources, P2 requires 3 resources, P3 requires 2 resources, P4 requires 9 resources, P5 requires 8 resources. What are the minimum no. of resources required so that system will never enter in deadlock?

Solution :
R=26 n=5
Minimum no. of resources required : 26-5+1=22

Maximum Processes For Safe State

If total no. of resources and need of each process are given, then no. of process can also be calculated. This can be calculated by using same concept.

(r-1)*n +1<=R where r is no. of resources required, n is no. of processes, R is total no. of resources.

Q. Suppose each process needs 5 units of resources and total no. of resources are 15. What are the maximum no. of process should run in safe state?

r=5, R=15
(5-1)*n+1<=15 => 4*n+1<=15
n<=14/4 => n<=3.5
So, no. of max. processes **n=3**

- Suppose each process needs 3 units of resources and total no. of resources are 100. What are the maximum no. of process should run in safe state?

R=100 r= 3
(r-1)*n +1<=R
(3-1)*n+1<=100
2n<=99 ; n<=49.5 that is, Maximum no. of processes must be 49

METHODS FOR HANDLING DEADLOCKS

Ensure that the system will *never* enter a deadlock state:

1. **Deadlock prevention**
2. **Deadlock avoidance**

Allow the system to enter a deadlock state and then recover. That is

3. **Deadlock detection and recovery**

Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX. That is

4. **Deadlock ignorance**

1. DEADLOCK PREVENTION

Restrain the ways request can be made:

1. **Mutual Exclusion** – not required for sharable resources (e.g., read-only files); must hold for non-sharable

resources

2. **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources. Require process to request and be allocated all its resources before it begins execution; Disadvantage:
   1) Low resource utilization
   Allow process to request resources only when the process has none allocated to it. (Release the acquired resource when requesting for additional resources;
   2) starvation possible
   3) Wait time out
3. **No Preemption** –
   If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released. Preempted resources are added to the list of resources for which the process is waiting. Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.
4. **Circular Wait** – impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration. A process can initially request any number of instances of a resource type -say, Ri .After that, the process can request instances of resource type Rj if and only if F(Rj) > F(Ri).

2. DEADLOCK AVOIDANCE

Safe State

When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state.

System is in safe state if there exists a sequence <P_1, P_2, …, P_n> of ALL the processes in the systems such that for each P_i, the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j, with $j < I$

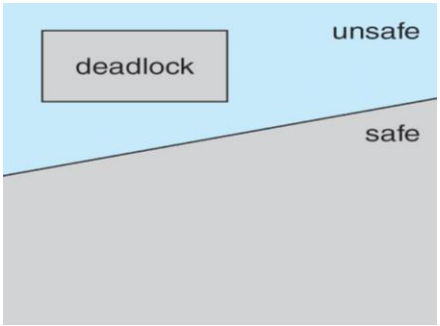
That is:
If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished.
When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate
When P_i terminates, P_{i+1} can obtain its needed resources, and so on.

Basic Facts

If a system is in safe state ⇒ no deadlocks

If a system is in unsafe state ⇒ possibility of deadlock

Avoidance ⇒ ensure that a system will never enter an unsafe state.



Safe, Unsafe, Deadlock State

Deadlock Avoidance Algorithms

- Single instance of a resource type : Use a resource-allocation graph
- Multiple instances of a resource type : Use the banker's algorithm

Deadlock Avoidance : Resource-Allocation-Graph Algorithm

- If we have a resource-allocation system with only one instance of each resource type, we can use a variant of the resource-allocation graph for deadlock avoidance.

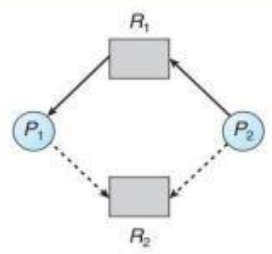
In addition to the request and assignment edges, we introduce a new type of edge, called a claim edge.

- A **claim edge** Pi --> Rj indicates that process Pi may request resource Rj at some time in the future.
- This edge resembles a request edge in direction but is represented in the graph by a dashed line.
- When process Pi requests resource Rj , the claim edge Pi --> Rj is converted to a request edge.
- When a resource Rj is released by Pi , the assignment edge Rj → Pi is reconverted to a claim edge Pi -> Rj

Suppose that process Pi requests resource Rj .

The request can be granted only if converting the request edge Pi → Rj to an assignment edge Rj → Pi does not result in the formation of a cycle in the resource-allocation graph.

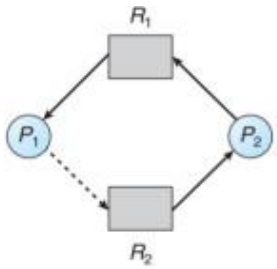
- If no cycle exists, then the allocation of the resource will leave the system in a safe state.
- If a cycle is found, then the allocation will put the system in an unsafe state



Resource-allocation graph for deadlock avoidance.

Suppose that P2 requests R2.

- Although R2 is currently free, we cannot allocate it to P2, since this action will create a cycle in the graph . A cycle, as mentioned, indicates that the system is in an unsafe state. If P1 requests R2, and P2 requests R1, then a deadlock will occur



An unsafe state in a resource-allocation graph.

DEADLOCK AVOIDANCE : BANKER'S ALGORITHM

- The resource-allocation-graph algorithm is not applicable to a resource allocation system with multiple instances of each resource type. Banker's algorithm is used in such situations.
- When a new process enters the system, it must declare the maximum number of instances of each resource type that it may need.
- This number may not exceed the total number of resources in the system.
- When a user requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state. If it will, the resources are allocated; otherwise, the process must wait until some other process releases enough resources.
- Several data structures must be maintained to implement the banker's algorithm.

Data Structures for the Banker's Algorithm

- **Available***:* Vector of length *m*. If available [*j*] = *k*, there are *k* instances of resource type R_j available
- **Max***: n x m* matrix. If *Max* [*i,j*] = *k*, then process P_i may request at most *k* instances of resource type R_j
- **Allocation***: n* x *m* matrix. If Allocation[*i,j*] = *k* then P_i is currently allocated *k* instances of R_j
- **Need***: n* x *m* matrix. If *Need*[*i,j*] = *k*, then P_i may need *k* more instances of R_j to complete its task.
- ***Need* [*i,j*] = *Max*[*i,j*] – *Allocation* [*i,j*]**

Safe Sequence:

[Max – Allocation]

	Allocation				Max				Available				Need			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0	0	0	0	0
P1	1	0	0	0	1	7	5	0	1	5	3	2	0	7	5	0
P2	1	3	5	4	2	3	5	6	2	8	8	6	1	0	0	2
P3	0	6	3	2	0	6	5	2	2	14	11	8	0	0	2	0
P4	0	0	1	4	0	6	5	6	2	14	12	12	0	6	4	2

Here the total resources are: A= Total allotted(A) +Total available(A)=2+1=3
Similarly find total of other resources.
Sometimes total resources will be given instead of available matrix.
In that case, we have to find available resource matrix as:

1. Find total number of allotted from allocation table for A, B,C and D
2. From total resource of A(given in question), subtract the total number allotted for A (calculated from step 1) to get available of A.
3. Similarly for all other resources.

Need = Max-Allocation
Check and compare Available and Need .**Safe sequence is P0 P2 P3 P4 P1**

Total			
A	B	C	D
3	14	12	12

1.Find Need Matrix.
2. Is system in a safe state?
3. If yes, find the safe sequence.

Banker's Algorithm(Safety Algorithm)

- **n - the number of processes in the system**
- **m - the number of resource types .**

1. ***Let Available and Flag be vectors of length m and n, respectively Flag [i] = false for i = 0, 1, …, n- 1***
   Find Need[n][m]=Max[n][m]-Allocation[n][m]

2. ***Find a process Pi such that both:***
   (a) Flag [i] = false
   (b) $Need_i \leq Available$
   If no such i exists, go to step 4

3. ***Available = Available +***
   $Allocation_i$ Flag [i] = true
   Go to step 2.Otherwise go to step 4.

If Flag [i] == true for all i, then the system is in a safe state; otherwise the system is in unsafe state.

Resource-Request Algorithm for Process P_i

$Request_i$ = request vector for process P_i. If $Request_i[j] = k$ then process P_i wants k instances of resource type R_j

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim

2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available

3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$Available = Available - Request_i;$$

$$Allocation_i = Allocation_i + Request_i;$$

$$Need_i = Need_i - Request_i;$$

If safe $\Rightarrow$ the resources are allocated to P_i

If unsafe $\Rightarrow$ P_i must wait, and the old resource-allocation state is restored

3. DEADLOCK DETECTION & RECOVERY

Deadlock Detection

If a system does not employ either a deadlock-prevention or a deadlock avoidance algorithm, then a deadlock situation may occur. In this environment, the system may provide:

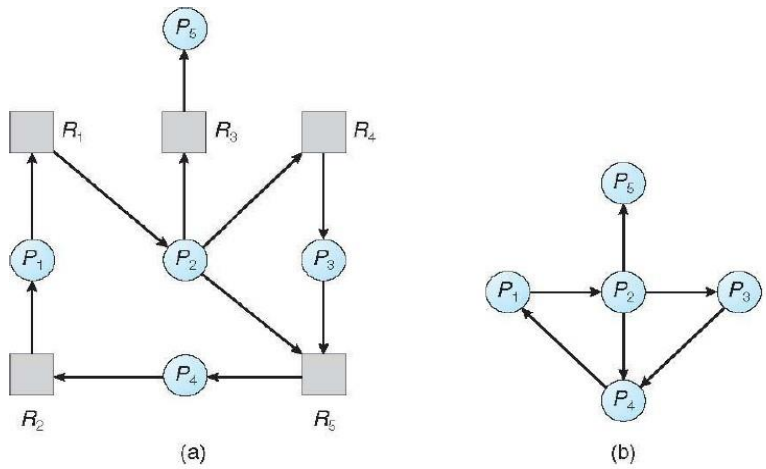
- An algorithm that examines the state of the system to determine whether a deadlock has occurred
- An algorithm to recover from the deadlock

Single and Multiple Instance of Each Resource Type

1. If all resources have only a single instance, then we can define a deadlock detection algorithm that uses a variant of the resource-allocation graph, called a wait-for graph. We obtain this graph from the resource-allocation graph by removing the resource nodes and collapsing the appropriate edges.

Resource-Allocation Graph and Wait-for Graph

- An edge from Pi to Pj in a wait-for graph implies that process Pi is waiting for process Pj to release a resource that Pi needs.
- An edge Pi → Pj exists in a wait-for graph if and only if the corresponding resource allocation graph contains two edges Pi → Rq and Rq → Pj for some resource Rq .
- A deadlock exists in the system if and only if the wait-for graph contains a cycle. To detect deadlocks, the system needs to maintain the wait for graph and periodically invoke an algorithm that searches for a cycle in the graph.



Resource-Allocation Graph

Corresponding wait-for graph

In Wait For Graph, the formation of cycle is the necessary and sufficient condition for the deadlock to occur

If the resources have multiple instances , use Banker's algorithm.

Example

- Consider a system with five processes P0 through P4 and three resource types A, B, and C. Resource type A has seven instances, resource type B has two instances, and resource type C has six instances.
- Suppose that, at time T0, we have the following resource-allocation state:

	Allocation	Request	Available
	A B C	A B C	A B C
• P0	0 1 0	0 0 0	0 0 0
• P1	2 0 0	2 0 2	
• P2	3 0 3	0 0 0	
• P3	2 1 1	1 0 0	
• P4	0 0 2	0 0 2	

- We claim that the system is not in a deadlocked state. If we execute our algorithm, we will find that the sequence <P0, P2, P3, P1, P4> results in Finish[i] == true for all i.
- Suppose now that process *P2 makes one additional request for an instance* of type *C. The Request matrix is modified as follows:*

Request

	A	*B*	*C*
P0	0	0	0
P1	2	0	2
P2	0	0	1
P3	1	0	0
P4	0	0	2

- We claim that the system is now deadlocked.

When multiple instances of resources present, formation of cycle is necessary but not sufficient condition for the deadlock to occur.

In deadlock detection and recovery, first we detect deadlock(let deadlock to occur) and then find ways to recover.

Deadlock Recovery

1. **Process termination**

a) Abort all deadlock process.
b) Abort 1 process at a time until deadlock cycle is eliminated
Process Termination
• To eliminate deadlocks by aborting a process, we use one of two methods. In both methods, the system reclaims all resources allocated to the terminated processes.
• Abort all deadlocked processes - The deadlocked processes may have computed for a long time, and the results of these partial computations must be discarded and probably will have to be recomputed later.
• Abort one process at a time until the deadlock cycle is eliminated - This method incurs considerable overhead, since after each process is aborted, a deadlock-detection algorithm must be invoked to determine whether any processes are still deadlocked.

2. **Resource Pre-emption**

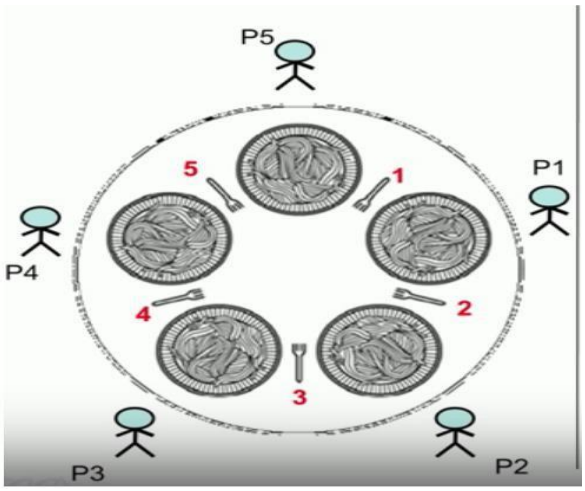
Resource pre-emption means releasing the resource

3 issues need to be considered
a) Selecting a victim.
b) Rollback
c) Starvation

To eliminate deadlocks using resource preemption, we successively preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken.
Three issues need to be addressed:
1. **Selecting a victim**. Which resources and which processes are to be preempted? As in process termination, we must determine the order of preemption to minimize cost
2. **Rollback**. If we preempt a resource from a process, what should be done with that process? Clearly, it cannot continue with its normal execution; it is missing some needed resource. We must roll back the process to some safe state and restart it from that state.
3. **Starvation**. How do we ensure that starvation will not occur? That is, how can we guarantee that resources will not always be preempted from the same process?

Syncronization problem 3: THE DINING-PHILOSOPHERS PROBLEM

- Consider five philosophers who spend their lives thinking and eating. The philosophers share a circular table surrounded by five chairs, each belonging to one philosopher.



- When a philosopher thinks, he does not interact with his colleagues.
- From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest to him (the chopsticks that are between him and his left and right neighbours).
- A philosopher may pick up only one chopstick at a time.
- Obviously, he cannot pick up a chopstick that is already in the hand of a neighbour.
- When a hungry philosopher has both his chopsticks at the same time, he eats without releasing the chopsticks.
- When he is finished eating, he puts down both chopsticks and starts thinking again.
- The dining-philosophers problem is considered a classic synchronization problem.
- It is a simple representation of the need to allocate several resources among several processes in a deadlock-free and starvation-free manner.

Problem Statement: Develop an algorithm where no philosopher starves.

- One simple solution is to represent each chopstick with a semaphore.
- A philosopher tries to grab a chopstick by executing a wait() operation on that semaphore. He releases his chopsticks by executing the signal() operation on the appropriate semaphores.

- **Thus, the shared data are semaphore chopstick[5];**

where all the elements of chopstick are initialized to 1

```
do {
   wait(chopstick[i]);
   wait(chopstick[(i+1) % 5]);
      . . .
   /* eat for awhile */
      . . .
   signal(chopstick[i]);
   signal(chopstick[(i+1) % 5]);
      . . .
   /* think for awhile */
      . . .
} while (true);
```

The structure of philosopher *i*.

Occurring Deadlock

- Suppose that all five philosophers become hungry at the same time and each grabs his left chopstick.
- All the elements of chopstick will now be equal to 0.
- When each philosopher tries to grab his right chopstick, he will be delayed forever.

Remedies to the deadlock problem :

- Allow at most four philosophers to be sitting simultaneously at the table.
- Allow a philosopher to pick up his chopsticks only if both chopsticks are available.
- Use an asymmetric solution—that is, an odd-numbered philosopher picks up first his left chopstick and then his right chopstick, whereas an even numbered philosopher picks up his right chopstick and then his left chopstick.

Solution with Semaphores

Uses N semaphores (s[1], s[2], …., s[N]) all initialized to 0, and a mutex
Philosopher has 3 states: HUNGRY, EATING, THINKING
A philosopher can only move to EATING state if neither neighbor is eating

```
void philosopher(int i){
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks();
    }
}
```

```
void take_forks(int i){
    lock(mutex);
    state[i] = HUNGRY;
    test(i);
    unlock(mutex);
    down(s[i]);
}
```

```
void put_forks(int i){
    lock(mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT)
    unlock(mutex);
}
```

```
void test(int i){
    if (state[i] = HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING){
        state[i] = EATING;
        up(s[i]);
    }
}
```

**********END**********